

“섬 (island)” 문제 풀이

작성자: 조승현

모든 입력에서, 지역 건설 연산은 1번만 사용하면 충분하며 1번은 반드시 필요하기 때문에 지역 건설을 정확히 1번 사용하는 방법을 찾으면 충분하다. 이에 대한 증명 역시 아래 풀이에 포함되어 있다.

부분문제 1

$N = 3, 4, 5$ 인 경우, 동일한 모양의 그래프를 하나로 칠 때 각 N 에 대해 한 종류의 그래프만이 존재한다 (다른 모든 지역과 연결된 지역을 생각하면 편하다). 해당 그래프에 대해서만 문제를 해결해서 답을 출력하면 된다. 구체적인 방법은 부분문제 1을 포함하는 부분문제 2의 풀이에 작성하였다.

부분문제 2

0번 지역이 $1, 2, \dots, N-1$ 번 지역과 연결되어 있다고 가정할 수 있다. 0, 1, 2번 지역에 대해 지역 건설을 사용하여 N 번 지역을 건설하면, 다음과 같은 간선 구성을 가지는 두 트리를 찾을 수 있다.

- $T_1 = \{(0, 2), (0, 3), \dots, (0, N-1), (0, N)\} \cup \{(1, N)\}$
- $T_2 = \{(0, 1), (1, 2), \dots, (N-2, N-1)\} \cup \{(2, N)\}$

부분문제 3

육지 도로 $N-3$ 개로 IOI 나라는 삼각분할되는 것을 생각하면 부분문제 3의 조건 하에서 육지 도로들은 사이클이 존재하지 않는 그래프를 이룬다는 것을 알 수 있다. 나아가, 육지 도로는 $N-2$ 개 지점에 대한 트리가 된다. 육지 도로의 트리가 포함하지 않는 두 지점을 A, B 번 지점이라 하자. A 번 지점은 $A+1, A-1$ 번 지점과만 연결되어 있고, $A+1$ 번 지점과 $A-1$ 번 지점은 육지 도로로 연결되어 있다. ($A=0$ 인 경우는 $1, N-1$ 번 정점, $A=N-1$ 인 경우는 $0, N-2$ 번 정점과 연결되어 있지만, 편의상 앞과 같이 서술하였다) $A, A+1, A-1$ 번 지점에 대해 지역 건설을 한 후에는 다음과 같은 두 트리를 찾을 수 있다.

- $T_1 = (\{(0, 1), (1, 2), \dots, (N-2, N-1), (N-1, 0)\} \cup \{(A, N), (A+1, N)\}) \setminus \{(A, A+1), (B, B+1)\}$
- $T_2 = E \setminus T_1$

위에서 T_1 은 최대한 해안 도로를 많이 사용, T_2 는 최대한 육지 도로를 많이 사용해서 만든 트리라고 생각하면 이해하기 쉽다.

부분문제 4

아무 지역 건설 가능한 점 a, b, c 를 골라 지역 건설을 시행하면 지역은 $N+1$ 개, 도로는 $2N$ 개가 된다. 또한, 2개 미만의 도로와 연결된 지역은 존재하지 않는다. 비둘기집의 원리에 의해 2개 이하의 도로와 연결된 지역이 존재하므로, 정확히 2개의 도로와 연결된 지역이 존재한다. 이를 u 라 하자. u 와 연결된 간선을 v, w 라 하면 v 와 w 는 도로로 연결되어 있다. 그래프에서 u 및 u 와 연결된 간선을 지우면 지역은 N 개, 도로는 $2N-2$ 개가 되고, 또한 이 그래프 역시 삼각분할 형태의 그래프가 되며, 앞서 서술한 조건을 만족함을 알 수 있다. 따라서, 문제의 크기가 $N+1$ 에서 N 으로 작아진 셈이 된다. 작아진 문제를 해결한 후 찾은 두 트리를 T_1, T_2 라 할 때, T_1 에 (u, v) 를, T_2 에 (u, w) 를 추가하면 원래 문제의 트리가 된다. 따라서, 차수가 2인 정점을

찾아 제거하고 작아진 문제를 해결한 후 두 트리에 간선을 하나씩 추가해주는 것을 반복하여 문제를 해결할 수 있다.

부분문제 5

부분문제 4에서 설명한 문제를 queue등의 자료구조를 이용하여 $O(N)$ 에 구현하면 부분문제 5를 해결할 수 있다.

“점프 게임 (jumpgame)” 문제 풀이

작성자: 구재현

부분문제 1

N 이 작기 때문에 배열 $A[i]$ 를 변숫값 배열을 사용하여 직접 계산할 수 있다. $f(i)$ 를 주인공이 i 번 발판에서 출발했을 때 얻을 수 있는 최대 점수라고 할 때, $f(i) = \max(f(i+K) + A[i], f(i+1))$ 이 성립한다. 이 동적 계획법 점화식을 $O(N)$ 에 구현하면, 부분문제 1을 통과할 수 있다.

부분문제 2

점프를 하더라도 다음 칸에 떨어지기 때문에 모든 발판의 점수를 얻을 수 있다. 고로 답은 모든 $A[i]$ 의 합이다. 이는 아래와 같이 계산할 수 있다.

```
return accumulate(R.begin(), R.end(), 0ll) - accumulate(L.begin(), L.end(), 0ll) + Q;
```

부분문제 4

우리가 해결하고자 하는 문제는, 서로 간격이 K 이상인 점들을 $[0, N-1]$ 에서 적절히 골라서, $A[i]$ 의 합을 최대화하는 것으로 해석할 수 있다. 합을 최대화하는 점들의 번호를 $i_1, i_2, \dots, i_m$ 라고 하자. 다음 사실이 성립한다:

Lemma. 모든 $1 \leq x \leq m$ 에 대해, $i_x = E[j] - tK$ 를 만족하는 어떠한 $0 \leq j \leq N-1, 0 \leq t$ 가 존재한다. (즉, i_x 는 $E[j], E[j] - K, E[j] - 2K, \dots$ 중 하나이다)

증명. 어떠한 해의 i_x 가 위 형태를 만족하지 않는다고 하자. 이 때 위 형태를 만족하지 않는 i_x 중 x 를 최대화하는 점을 고른다.

- $x = m$ 인 경우, i_x 를 포함하는 구간이 존재한다면, 이들 중 끝점이 최소인 구간의 끝점으로 옮긴다. 포함하는 구간이 없다면 아예 점을 지워도 된다.
- $x < m$ 인 경우, $i_{x+1} = E[j] - tK$ 가 성립하며, 고로 가정에 의해 $i_x < E[j] - (t+1)K$ 이다. i_x 를 포함하고 $E[j] - (t+1)K$ 를 포함하지 않는 구간이 존재한다면, 이들 중 끝점이 최소인 구간의 끝점으로 옮긴다. 포함하는 구간이 없다면, $i_x = E[j] - (t+1)K$ 로 설정해 주면 된다.

이를 반복하면, 모든 해가 위 조건을 만족하게 할 수 있다. ■

부분문제 4에서는, K 가 크기 때문에 $E[j] - tK$ 꼴의 점이 최대 $5Q$ 개이다. 위와 같은 점들의 정렬된 리스트를 $Q_1, Q_2, \dots, Q_p$ 라고 하자. 이 점들만을 상태로 가지는, 다음과 같은 동적 계획법을 사용한다. $DP[i]$ 를, 서로 간격이 K 이상인 점들을 $Q_1, Q_2, \dots, Q_i$ 에서 적절히 골라서 얻을 수 있는 $A[Q_i]$ 합의 최대라고 하면, 다음 식이 성립한다 ($Q_0 = -\infty$ 로 두자):

$$DP[i] = \max(DP[i-1], DP[j] + A[Q_i])$$

$$\text{where } j = \max_{0 \leq k \leq i-1, Q_k \leq Q_i - K} k$$

이 동적 계획법 점화식을 계산하면 문제를 해결할 수 있다. 하지만 그대로 계산하면 비효율적일 수 있어 다음과 같은 최적화가 필요하다.

- 첫 번째 문제는 $A[Q_i]$ 를 계산하는 것이다. 부분문제 1에서 사용한 변숫값 배열을 똑같이 사용하는데, 증가시키는 위치를 이분 탐색으로 찾아주면 된다.
- 두 번째 문제는 j 를 계산하는 것이다. Two pointers를 사용하면 된다.

두 점을 고려해주면 부분문제 4를 $O(\frac{N}{K}Q \log Q)$ 에 해결할 수 있다.

부분문제 6

설명상 편의를 위해 부분문제 1의 동적 계획법을 다음과 같이 변형해서 사용한다. 부분문제 4의 관찰에 의해, 이렇게 변형하여도 답이 바뀌지 않는다.

$$f(i) = \max(f(i - K) + A[i], f(i - 1))$$

관찰은, $f(i)$ 를 계산하는 과정을 1차원 DP의 형태로 보지 않고 2차원 DP의 형태로 보는 것이다. $i = qK + r$ ($0 \leq r \leq K - 1$, q, r 은 음이 아닌 정수) 라고 하면, 위 점화식을 다음과 같이 표현할 수 있다. 편의상 N 이 K 의 배수라고 가정하자.

$$f(q, 0) = \max(f(q - 1, 0) + A[q][0], f(q - 1, K - 1))$$

$$f(q, r) = \max(f(q - 1, r) + A[q][r], f(q, r - 1))$$

점화식을 위와 같이 표현할 경우, 두 가지 점을 활용할 수 있다. 첫 번째로, 부분문제 4에서 사용한 Lemma에 따라서, r 은 $E[j] \pmod{K}$ 중 하나고, 고로 r 의 후보가 $O(N)$ 개로 줄어든다. 두 번째로, $A[q][r]$ 의 형태가 보통은 그렇게 복잡하지 않다. 예를 들어, 대다수의 행 q 에 대해서 $A[q][r]$ 의 값은 모두 동일하다. 각 구간이 추가될 때마다 위 조건이 만족하지 않게 되는 행이 최대 두 개이기 때문이다. 구체적으로, 입력으로 받은 쿼리 구간들을 스위핑하듯이 처리하여 $A[i]$ 가 같은 구간끼리 묶게 되면, 최대 $2Q$ 개의 구간이 나온다. 만약 한 구간의 길이가 충분히 크다면, 이 구간은 여러 행을 전부 같은 값으로 덮게 된다.

각 행을 두 가지 종류로 구분하자. 첫 번째 종류는, 해당 행에 서로 다른 $A[q][r]$ 값이 존재하는 경우고, 두 번째 종류는, 해당 행의 모든 $A[q][r]$ 값이 같은 경우이다. 첫 번째 종류는 최대 $2Q$ 번 등장하니, $f(q, r)$ 을 배열 등으로 관리한다면 단순히 $O(Q)$ 정도에 계산해 줄 수 있다. $Q \leq 5000$ 이라 이렇게 하여도 시간 복잡도가 $O(Q^2)$ 가 되어 시간 안에 동작한다.

두 번째 종류는, 해당 행의 모든 $A[q][r]$ 값이 같은 경우이다. 이 경우, 이 행을 완전히 포함하는 구간을 생각해 보면, 이 행 뿐만 아니라 이 행과 인접한 몇 개의 구간들에 대해서 $A[q][r]$ 값이 모두 같음을 알 수 있다. 고로 문제를 다음과 같은 식으로 환원할 수 있다.

- i 에 따라 증가하는 함수 $f(0, i)$ 가 있다. 모든 $A[i][j]$ 의 값이 X 일 때, $f(T, i)$ 함수를 계산하여라.

이 문제는 T 의 크기에 상관없이 $O(Q)$ 시간에 해결할 수 있다. $f(1, i)$ 를 단순히 정의에 따라서 계산하면, $f(1, K - 1) - f(1, 0) \leq X$ 가 성립한다. 고로 이 시점부터는 $f(q, i) = f(q - 1, i) + X$ 의 형태로 점화식을 단순화할 수 있고, 이렇게 되면 단순히 구간에 $(T - 1)X$ 를 더해주면 되기 때문이다.

위 방법을 사용하면 $f(\frac{N}{K}, i)$ 를 모두 계산할 수 있고, 문제의 정답 역시 계산할 수 있다. 시간 복잡도는 $O(Q^2)$ 이다.

부분문제 7

$f(*, i)$ 를 효율적으로 관리하는 자료구조를 구성하고, 부분문제 6의 풀이를 사용하면 만점을 받을 수 있다. 함수 형태를 효율적인 자료구조로 관리하여 동적 계획법을 최적화한다는 점에서, Slope trick을 사용한 풀이라고 생각할 수도 있다.

자료구조는, 다음과 같은 정보를 관리한다:

- $f(i, 0)$ 의 값 (L)
- $f(i, K - 1)$ 의 값 (R)
- $f(i, j) \neq f(i, j + 1)$ 일 경우, $(j, f(i, j + 1) - f(i, j))$ 의 쌍 (D)

세 번째 값 (D) 은 `std::map` 과 같은 자료구조로 관리한다.

두 번째 종류에서 나오는, 전체 $f(i, j)$ 에 특정 수 X 를 더하는 것은 L, R 을 조정하는 것으로 $O(1)$ 시간에 해결할 수 있다. 첫 번째 종류의 쿼리는, 구간에 특정 수를 더한 후, 구간 오른쪽에서 $f(i, j) > f(i, j + 1)$ 인 지점이 발생하면 이 부분을 $f(i, j)$ 로 맞춰주어 정렬 조건을 유지하는 것임을 관찰할 수 있다. 만약 수를 더하는 구간이 $f(i, *)$ 의 Suffix라면, 연산 이후에도 정렬성이 깨지지 않아 D 에 값 하나를 추가하는 식으로 쉽게 구현할 수 있다. Suffix가 아닌 경우에는 쿼리의 결과가 $f(i, j)$ 의 정렬성을 깨서 문제가 되는데, 구간의 오른쪽 끝에서부터 순서대로 D 의 값을 제거하면서 정렬성을 보장하는 식으로 구현할 수 있다. 각 쿼리에서 D 에 삽입하는 값의 수가 $O(1)$ 개이기 때문에 총 $O(Q \log Q)$ 시간 복잡도에 문제를 해결할 수 있다.

“팰린드롬 판별하기 (palindrome)” 문제 풀이

작성자: 채이환

15점

`find_character` 함수에서 $Y = \{y\}$ 라면 $S[x]$ 와 $S[y]$ 가 같은지 비교하는 용도로 사용할 수 있다. 이 함수를 N 번까지 사용할 수 있으므로 서로 대칭인 위치에 있는 문자가 같은지 모두 확인하여 팰린드롬인 판별할 수 있다.

50점

우선 `find_character` 함수로 $S[0]$ 과 $S[N-1]$ 이 같은지 판별한다. 다르다면 팰린드롬이 아니고, 같다면 다음 단계로 넘어간다.

`count_pair(0, i, N-1)`, `count_pair(0, N-i-1, N-1)`을 질의한다.

- $S[0]$ 과 $S[N-1]$ 이 같으므로 결과가 0이 나올 수는 없다.
- 두 결과가 다르면 $S[i]$ 와 $S[N-i-1]$ 이 다르므로 팰린드롬이 아니다.
- 두 결과가 모두 3이라면 $S[i]$ 와 $S[N-i-1]$ 이 같음을 확인했으므로 넘어가도 좋다.
- 두 결과가 모두 1이라면, `count_pair(0, i, N-i-1)`에서 1이 나오면 팰린드롬, 0이 나올 경우 팰린드롬이 아님을 알 수 있다.

이 과정을 반복하면 대략 $1.5N$ 번의 `count_pair`을 사용하게 되어, 50점을 받을 수 있다.

만점

$1 \leq i < \lfloor \frac{N}{2} \rfloor$ 에 대해 `count_pair(0, i, N-1-i)`을 사용한다.

- 0이 나올 경우 팰린드롬이 아니다.
- 3이 나올 경우 넘어가도 좋다.
- 1이 나왔을 때는 집합 Y 에 $i, N-i-1$ 을 추가한다.

위 호출에서 1이 나온 경우에는 $S[0]$ 과 $S[i]$, $S[N-i-1]$ 둘 중 하나가 같을 경우와 $S[i]$, $S[N-i-1]$ 이 같고 $S[0]$ 이 다른 경우가 있다. 전자의 경우 팰린드롬이 아니다. 위 경우를 확인하기 위해 `find_character(0, Y)`를 사용한다. 1이 나오면 팰린드롬이 아니고, 0이 나오면 다음 단계로 넘어간다.

이제 확인할 것은 $S[0]$ 과 $S[N-1]$ 이 같은지의 여부이다. 50점 풀이에서 했던 방식과 같이, 최대 3번의 질의를 통해 확인할 수 있다.

위 방법과 같은 가장 효율적인 방법을 사용한다면 `count_pair`를 $\lfloor \frac{N}{2} \rfloor$ 번, `find_character`을 1번 사용하므로 만점을 받을 수 있다. 같은 아이디어를 채택했지만 풀이에서 `count_pair`을 비효율적으로 더 사용하는 경우가 있다면 70점 또는 90점을 받을 수 있다.

“바이러스 (virus)” 문제 풀이

작성자: 구재현

부분문제 1

문제에서 바이러스가 전파 되는 과정을 최단 경로 문제로 해석하면 이후 풀이 이해와 구현에 도움이 된다. j 번 사람의 영역 을 $P[j]$ 에서 거리가 $D[j]$ 이하인 지점의 집합이라고 하자. i 번 사람과 j 번 사람 간의 전파의 매개체는, i 번 사람과 j 번 사람의 영역의 교집합이다. $cost(i, j)$ 를, 이 교집합에 속하는 점 v 중 $C[v]$ 의 최솟값이라고 하자 (없을 경우 ∞).

i 번 사람이 시간 t 에 감염되면, j 번 사람은 시간 $t + cost(i, j)$ 에 바이러스에 감염되어 있다. 각 사람을 정점으로 하고, i 번 사람에서 j 번 사람으로 가는 가중치 $cost(i, j)$ 의 간선을 가지는 그래프를 G 라고 하자. 바이러스가 전파되는 방식은, 0 번 정점에서 시작하는 최단 경로를 따른다.

고로 이 문제는 위에서 설명한 방식으로 그래프를 만든 후 Dijkstra's algorithm 이나 Floyd-Warshall algorithm 등을 사용하여 0 번 정점에서 각 정점에 도달하는 최단 경로를 계산하는 문제라고 생각할 수 있다. 그래프를 만들기 위해서는 $cost(i, j)$ 를 계산해야 하는데, 지문의 정의를 따라서 계산해 주면 된다. 시간 복잡도는 $O(N^2(N + M))$ 정도이다.

부분문제 2

부분문제 2에서는 그래프 크기가 크기 때문에 Floyd-Warshall algorithm은 사용할 수 없으며, $cost(i, j)$ 역시 단순하게 계산할 수 없다. $cost(i, j)$ 가 배열의 구간 최솟값 형태로 표현되기 때문에, 적절한 전처리를 사용하여 빠르게 해결할 수 있다. 위 방법을 사용해도 부분문제 2를 맞을 수 있지만, 이후 부분 점수로 확장이 편리한 다른 풀이를 소개한다.

바이러스가 전파되는 과정을 모델링할때, 지점 역시 감염될 수 있다고 생각하고, 사람에 대한 정점 뿐만이 아니라 지점에 대한 정점을 추가로 만들어주자. 만약 i 번 사람이 시간 t 에 감염되었다면, i 번 사람의 영역에 속하는 모든 지점 v 가 시간 $t + C[v]$ 에 감염된다고 하자. 어떠한 지점이 감염된다면, 이 지점을 영역으로 가지는 모든 사람이 감염된다는 것을 관찰할 수 있다.

이렇게 그래프를 구성할 경우 정점의 개수는 $O(N + M)$, 간선의 개수는 $O(NM)$ 개로 약간 증가하지만, 각 간선의 가중치를 계산하기 위해 추가적인 작업이 필요하지 않다. 이 그래프에서 Dijkstra's algorithm을 적용하면 된다.

부분문제 3

부분문제 3에서는 KOI 도시의 구조가 직선의 형태고, 각 사람의 영역은 이 직선의 구간을 이룬다. 이 점을 활용하여, 같은 최단 경로를 조금 더 효율적인 그래프 구성으로 계산한다.

부분문제 2에서 만든 그래프는 다음과 같은 두 종류의 정점과, 두 종류의 간선이 있다.

- 정점 p_i 는 i 번 사람에 대응
- 정점 l_j 는 j 번 지점에 대응
- p_i 에서, i 번 사람의 지점 구간 l_j ($j \in [L_i, R_i]$) 로 가는 가중치 $C[j]$ 의 간선
- i 번 사람의 지점 구간 l_j ($j \in [L_i, R_i]$) 에서, p_i 로 가는 가중치 0 의 간선

세 번째 종류의 간선은 끝점에 따라 가중치가 다르기 때문에 그래프 구성을 효율화하기 어렵다. 고로 다음과 같이 그래프 구성을 약간 바꾼다.

- 정점 p_i 는 i 번 사람에 대응
- 정점 in_j, out_j 는 j 번 지점에 대응
- p_i 에서, i 번 사람의 지점 구간 in_j ($j \in [L_i, R_i]$) 로 가는 가중치 0 의 간선
- i 번 사람의 지점 구간 out_j ($j \in [L_i, R_i]$) 에서, p_i 로 가는 가중치 0 의 간선
- in_j 에서 out_j 로 가는 가중치 $C[j]$ 의 간선

구간에 간선을 더하는 것과 구간 쿼리가 유사하다는 것에서 착안해, 세그먼트 트리 형태로 그래프의 크기를 줄이자. n 개의 지점에 대해 세그먼트 트리를 만들고, 각 트리의 노드에 대응되는 두 개의 정점을 만든다. 모든 간선의 가중치는 0 이다.

- 세그먼트 트리의 각 노드 n 에 대해서 $INTREE_n, OUTTREE_n$ 이라는 정점을 만든다.
- 리프가 아닌 노드 n 에 대해, $INTREE_n \rightarrow INTREE_{2n}, INTREE_n \rightarrow INTREE_{2n+1}$ 의 형태로 간선을 만든다. $OUTTREE_n$ 에 대해서는 거꾸로 한다.
- 리프 노드 n 에 대해, 해당 노드에 대응되는 사람이 j 라고 하면, $INTREE_n \rightarrow in_j, out_j \rightarrow OUTTREE_n$ 형태의 간선을 만든다.

이 세그먼트 트리는 $O(n)$ 개의 정점과 $O(n)$ 개의 간선을 가진다. 이 세그먼트 트리 구조를 사용하여, 그래프의 간선 수를 줄일 수 있다: p_i 에서, i 번 사람의 지점 구간 in_j ($j \in [L_i, R_i]$) 로 가는 가중치 0 의 간선을 만드는 대신, 세그먼트 트리에서 해당 구간에 대응되는 $O(\log n)$ 개의 노드에 대해, $INTREE_n$ 으로 가는 가중치 0 의 간선을 만들면 된다. $OUTTREE_n$ 에 대해서도 동일하게 진행하면, 총 $O(n+m)$ 개의 정점과 $O(n + m \log n)$ 개의 간선을 가지는 그래프를 얻게 된다. 이 그래프에서 Dijkstra's algorithm을 사용하면 전체 문제를 $O(n \log n + m \log^2 n)$ 에 해결할 수 있다.

추가적으로, 가중치가 0 이 아닌 간선이 최대 n 개라는 점에 착안하여, Dijkstra's algorithm을 처리하는 과정에서 0 인 간선들이 존재할 때는 BFS로 처리하는 방법도 존재한다. 0-1 BFS와 비슷한 원리이며, 이 방법을 사용하면 전체 문제를 $O((n+m) \log n)$ 에 해결할 수도 있다. 실제 수행 시간은 대략 2배 정도 빨라지며, 이 최적화를 적용하지 않아도 만점을 받을 수 있게 시간 제한이 설정되어 있다.

부분문제 4

부분문제 1의 풀이를 트리에서 구현하면 된다. 트리에서 모든 점간 최단 경로를 계산해 두면 가능하다.

부분문제 5

부분문제 2의 풀이를 트리에서 구현하면 된다. $cost(i, j)$ 를 $O(1)$ 에 구하는 방식은 일반화가 (가능하지만) 다소 복잡하고, 이후 설명한 다른 풀이를 적용하는 것이 좋다.

부분문제 6

부분문제 3의 풀이를 트리에서 구현하기 위해서는, 트리에서 특정 정점과 거리가 D 이하인 정점들의 집합을 효율적으로 표현할 수 있어야 한다. 부분문제 3에서 사용한 세그먼트 트리로는 이것이 어렵지만, 대신 Centroid decomposition tree를 사용하면 이러한 표현이 가능하다.

트리의 Centroid decomposition을 할 때, 우리는 주어진 트리의 Centroid c 를 찾고, c 를 루트로 한 트리에 대해 적절한 전처리를 한 후, c 를 제거한 각 서브트리들에 대해 재귀적으로 문제를 해결한다. 트리의 각 정점 v 에 대해서, v 가 Centroid였을 시점에 v 를 포함한 트리를 v 의 클러스터라고 하자. 또한, 각 정점 v 에 대해서 Centroid decomposition tree는, 트리에서 Centroid decomposition을 했을 때 특정 정점 v 와 거리가 D 이하인 정점들은 다음 집합의 합집합이다:

- v 의 클러스터 상에서 v 와의 거리가 D 이하
- $p(v)$ 의 클러스터 상에서 $p(v)$ 와의 거리가 $D - \text{dist}(p(v), v)$ 이하
- $p^2(v)$ 의 클러스터 상에서 $p^2(v)$ 와의 거리가 $D - \text{dist}(p^2(v), v)$ 이하
- ...

각 클러스터 c 에 대해, $\text{INDECOMP}_{c,d}$ 는 c 의 클러스터 상에서 c 와의 거리가 d 이하인 점들의 in_v 로 가는 정점으로 정의된다. $\text{OUTDECOMP}_{c,d}$ 역시 유사하게 정의한다 (방법이 동일하니 여기서부터는 설명하지 않는다.) 이 때 다음과 같이 간선을 이어주면 된다:

- $\text{INDECOMP}_{c,d} \rightarrow \text{INDECOMP}_{c,d-1}$
- c 의 클러스터 상에서 거리가 정확히 d 인 모든 정점 v 에 대해 $\text{INDECOMP}_{c,d} \rightarrow in_v$

이를 통해 총 $O(n \log n + m)$ 개의 정점과 $O((n + m) \log n)$ 개의 간선을 가지는 그래프를 얻게 된다. 이 그래프에서 Dijkstra's algorithm을 사용하면 전체 문제를 $O((n + m) \log^2 n)$ 에 해결할 수 있다. 부분문제 3과 동일한 최적화를 적용하면 이를 $O((n + m) \log n)$ 에 해결할 수도 있다.